

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра	<u>О7</u>	<u>Информационные системы и программная инженерия</u>
	шифр	наименование кафедры, по которой выполняется работа
Дисциплина	<u>Базы данных</u>	
		наименование дисциплины

ПРАКТИЧЕСКОЕ ЗАДАНИЕ

4

номер задания (при наличии)

Основы SQL. Операции с БД

Вариант 20 – «Кинотеатр»

при наличии указать тему учебно-практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ

группы О717Б

Праудин Д.С.

подпись

фамилия и инициалы

дата сдачи

ПРОВЕРИЛ

ученая степень, ученое звание, должность

Заборовский И. С.

подпись

фамилия и инициалы

Оценка / балльная оценка _____

дата проверки

Санкт-Петербург

20 24 г.

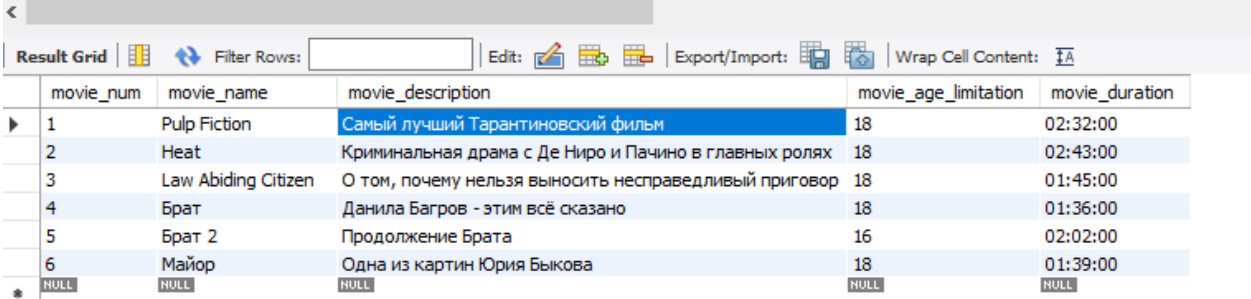
ПОСТАНОВКА ЗАДАЧИ

Целью данной работы является проектирование соответствующих запросов к БД, создаваемой в соответствии с индивидуальным заданием.

1 Выборка данных. Команда SELECT

Для получения всех объектов таблицы вместо указания конкретных столбцов вводится символ звёздочка – *. Результат выполнения команды SELECT с выводом всех столбцов продемонстрирован на рисунке 1 на примере таблицы «Movies» базы данных «mydb», которая была создана в результате выполнения 3 работы.

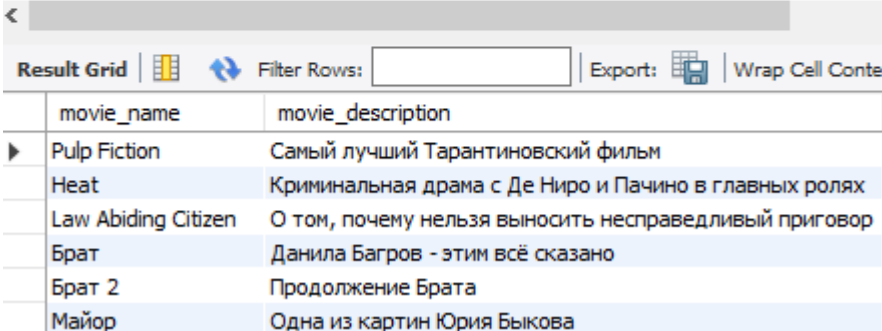
```
1 • USE mydb;
2
3 • SELECT * FROM `Movies`
4
5
6
7
8
```



movie_num	movie_name	movie_description	movie_age_limitation	movie_duration
1	Pulp Fiction	Самый лучший Тарантиновский фильм	18	02:32:00
2	Heat	Криминальная драма с Де Ниро и Пачино в главных ролях	18	02:43:00
3	Law Abiding Citizen	О том, почему нельзя выносить несправедливый приговор	18	01:45:00
4	Брат	Данила Багров - этим всё сказано	18	01:36:00
5	Брат 2	Продолжение Брата	16	02:02:00
6	Майор	Одна из картин Юрия Быкова	18	01:39:00
NULL	NULL	NULL	NULL	NULL

Рисунок 1 – Команда SELECT, вывод всех столбцов таблицы
Вывод конкретных столбцов таблицы представлен на рисунке 2.

```
1 -- USE mydb;
2
3 • SELECT movie_name, movie_description FROM `Movies`
4
5
6
7
8
```

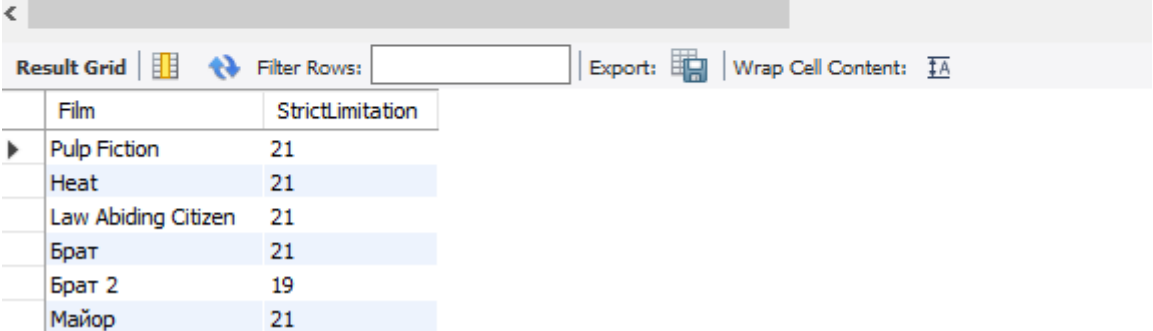


movie_name	movie_description
Pulp Fiction	Самый лучший Тарантиновский фильм
Heat	Криминальная драма с Де Ниро и Пачино в главных ролях
Law Abiding Citizen	О том, почему нельзя выносить несправедливый приговор
Брат	Данила Багров - этим всё сказано
Брат 2	Продолжение Брата
Майор	Одна из картин Юрия Быкова

Рисунок 2 – Команда SELECT, вывод указанных столбцов

Создание составного столбца и определение псевдонимов столбцов представлено на рисунке 3.

```
3 • SELECT movie_name AS Film, movie_age_limitation + 3 AS StrictLimitation
4 FROM `Movies`;
5
6
7
8
```



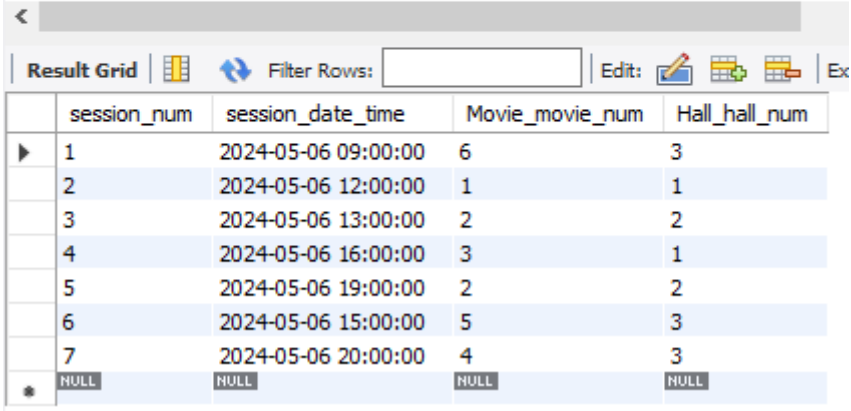
Film	StrictLimitation
Pulp Fiction	21
Heat	21
Law Abiding Citizen	21
Брат	21
Брат 2	19
Майор	21

Рисунок 3 – Команда SELECT, добавление псевдонимов столбцам и создание составного столбца

2 Фильтрация данных. Оператор WHERE

Результат выборки всех сеансов просмотра фильмов, дата и время которых меньше указанных, представлен на рисунке 4.

```
3 • SELECT * FROM `Sessions`
4 WHERE session_date_time <= '2024-05-07 00:00:00';
5
6
7
8
```



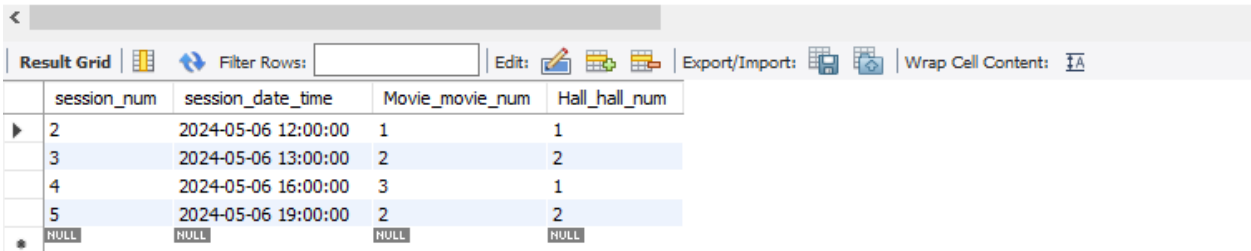
session_num	session_date_time	Movie_movie_num	Hall_hall_num
1	2024-05-06 09:00:00	6	3
2	2024-05-06 12:00:00	1	1
3	2024-05-06 13:00:00	2	2
4	2024-05-06 16:00:00	3	1
5	2024-05-06 19:00:00	2	2
6	2024-05-06 15:00:00	5	3
7	2024-05-06 20:00:00	4	3
*	NULL	NULL	NULL

Рисунок 4 – Выборка всех сеансов, дата и время которых меньше указанных

Логические операторы и приоритет операций

С помощью логических операторов можно осуществлять выборку по нескольким условиям. Вывод содержимого таблицы «Sessions» с ограничением не только по дате и времени, но и по индексу зала и фильма(индекс фильма – 1 или 2 или номер зала – 1) представлен на рисунке 5.

```
3 • SELECT * FROM `Sessions`
4 WHERE session_date_time <= '2024-05-07 00:00:00' AND (Movie_movie_num <= 2 OR Hall_hall_num = 1);
5
6
7
8
```



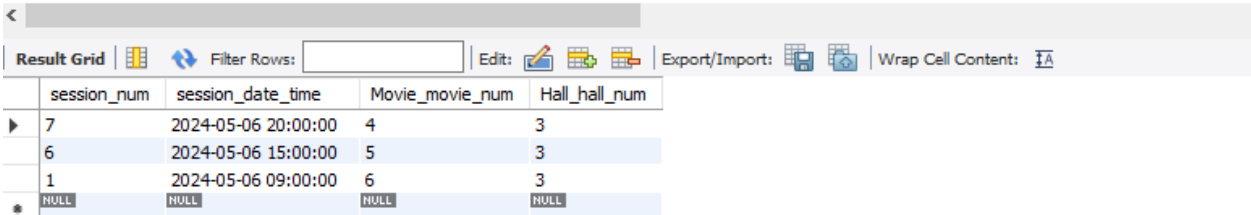
session_num	session_date_time	Movie_movie_num	Hall_hall_num
2	2024-05-06 12:00:00	1	1
3	2024-05-06 13:00:00	2	2
4	2024-05-06 16:00:00	3	1
5	2024-05-06 19:00:00	2	2
NULL	NULL	NULL	NULL

Рисунок 5 – Выборка с составным условием

В данном случае скобки, в которые обёрнуты операнды оператора OR переопределяют приоритет операций(сначала выполняется OR в скобках, а после неё AND, без скобок было бы наоборот).

С помощью оператора NOT можно реализовать инверсию составного условия целиком, что может быть полезно в некоторых случаях. Предыдущее ограничение на «индекс фильма – 1 или 2 или номер зала – 1» можно заменить на «индекс фильма > 2 и номер зала не равен 1» оборачиванием этого условия в скобки и добавлением перед ним ключевого оператора NOT, что представлено на рисунке 6.

```
3 • SELECT * FROM `Sessions`
4 WHERE session_date_time <= '2024-05-07 00:00:00' AND NOT(Movie_movie_num <= 2 OR Hall_hall_num = 1);
5
6
```



session_num	session_date_time	Movie_movie_num	Hall_hall_num
7	2024-05-06 20:00:00	4	3
6	2024-05-06 15:00:00	5	3
1	2024-05-06 09:00:00	6	3
NULL	NULL	NULL	NULL

Рисунок 6 – Добавление оператора NOT

3 Обновление данных. Команда Update

Перед изменением таблиц во избежание ошибки, представленной на рисунке 7, необходимо отключить безопасный режим снятием флажка у соответствующей настройки, что представлено на рисунке 8.

Error Code: 1175. You are using safe update mode and you tried to update a table without ... 0.000 sec

Рисунок 7 – Ошибка при попытке изменения таблицы в безопасном режиме

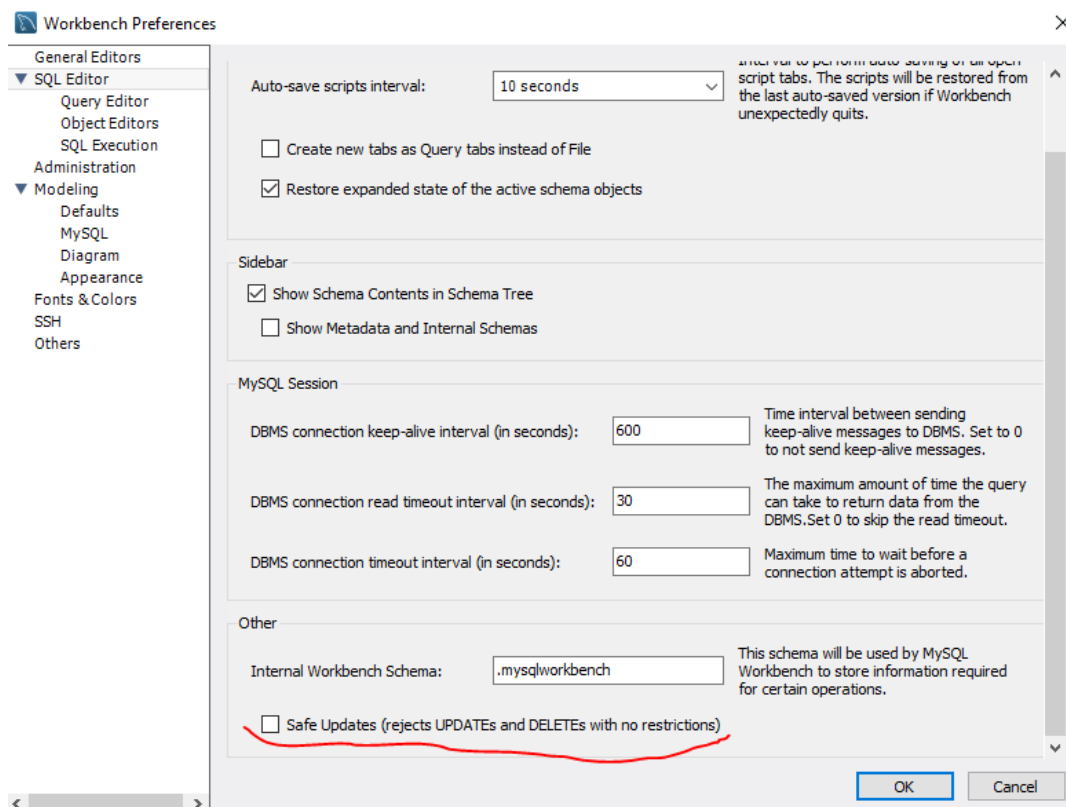


Рисунок 8 – Выключение безопасного режима

Данные таблицы «Tickets» до обновления представлены на рисунке 9.

```
3 • select * from Tickets;
4
5
```

ticket_row	ticket_seat	ticket_cost	Client_client_num	Cashier_cashier_num	Sessi
1	2	250	3	2	3
2	3	300	2	1	1
3	10	350	4	1	12
4	5	300	1	1	1
6	12	300	4	2	8
*	NULL	NULL	NULL	NULL	NULL

Рисунок 9 – Таблица «Tickets» до изменений

Уменьшение стоимости билета и изменение индекса кассира этого билета при условии, что у билета индекс кассира был равен 2, в таблице «Tickets» представлено на рисунке 10.

```

4 • UPDATE `Tickets`
5   SET Cashier_cashier_num = 1,
6       ticket_cost = ticket_cost - 70
7   WHERE Cashier_cashier_num = 2;
8
9 • select * from Tickets;
10

```

	ticket_row	ticket_seat	ticket_cost	Client_client_num	Cashier_cashier_num	Session_session_num
▶	1	2	180	3	1	3
	2	3	300	2	1	1
	3	10	350	4	1	12
	4	5	300	1	1	1
	6	12	230	4	1	8
*	NULL	NULL	NULL	NULL	NULL	NULL

Рисунок 10 – Изменение записей в таблице «Tickets»

4 Удаление данных. Команда DELETE

Содержимое таблицы «Sessions» до изменений представлено на рисунке 11.

```

4 • select * from Sessions;
5
6

```

	session_num	session_date_time	Movie_movie_num	Hall_hall_num
▶	1	2024-05-06 09:00:00	6	3
	2	2024-05-06 12:00:00	1	1
	3	2024-05-06 13:00:00	2	2
	4	2024-05-06 16:00:00	3	1
	5	2024-05-06 19:00:00	2	2
	6	2024-05-06 15:00:00	5	3
	7	2024-05-06 20:00:00	4	3
	8	2024-05-07 09:00:00	4	1
	9	2024-05-07 11:00:00	5	2
	10	2024-05-07 12:00:00	6	3
	11	2024-05-07 14:00:00	3	2
	12	2024-05-07 15:30:00	4	1
	13	2024-05-07 16:45:00	1	3
	14	2024-05-07 18:15:00	2	3
	15	2024-05-07 19:00:00	1	1
*	NULL	NULL	NULL	NULL

Рисунок 11 – Изначальное содержимое таблицы «Sessions»

Решено было удалить все сеансы, начинающиеся после 14:00 06.05.2024 и имеющие индекс кинокартины 1 или 4. Результат удаления представлен на рисунке 12.

```

4 • DELETE FROM Sessions
5   WHERE session_date_time > '2024-05-06 14:00:00' AND (Movie_movie_num = 1 OR Movie_movie_num = 4);
6 • select * from Sessions;
7

```

session_num	session_date_time	Movie_movie_num	Hall_hall_num
1	2024-05-06 09:00:00	6	3
2	2024-05-06 12:00:00	1	1
3	2024-05-06 13:00:00	2	2
4	2024-05-06 16:00:00	3	1
5	2024-05-06 19:00:00	2	2
6	2024-05-06 15:00:00	5	3
9	2024-05-07 11:00:00	5	2
10	2024-05-07 12:00:00	6	3
11	2024-05-07 14:00:00	3	2
14	2024-05-07 18:15:00	2	3
* NULL	NULL	NULL	NULL

Рисунок 12 – Содержимое таблицы «Sessions» после удаления

5 Неявное соединение таблиц

Были рассмотрены 5 таблиц: «Tickets», «Sessions», «Cashiers», «Clients» и «Movies».

Цель – получить полную информацию о билете.

Из всех таблиц была получена и слита в общую таблицу следующая информация: название фильма, дата и время сеанса, номер кинозала, номер ряда, номер места, стоимость билета, ФИО клиента, ФИО кассира.

Для сокращения кода запроса были использованы псевдонимы. Запрос и результат приведены на рисунке 13.

```

4 • SELECT M.movie_name, S.session_date_time, S.Hall_hall_num,
5   T.ticket_row, T.ticket_seat, T.ticket_cost, CL.client_full_name, CA.cashier_full_name
6   FROM Tickets as T, Sessions as S, Cashiers as CA, Clients as CL, Movies as M
7   WHERE T.Client_client_num = CL.client_num
8   AND T.Cashier_cashier_num = CA.cashier_num
9   AND T.Session_session_num = S.session_num
10  AND S.Movie_movie_num = M.movie_num;
11

```

movie_name	session_date_time	Hall_hall_num	ticket_row	ticket_seat	ticket_cost	client_full_name	cashier_full_name
Майор	2024-05-06 09:00:00	3	4	5	300	Петр Иванович Сидоров	Балембин Кирилл Саныч
Майор	2024-05-06 09:00:00	3	2	3	300	Сидор Петрович Иванов	Балембин Кирилл Саныч
Heat	2024-05-06 13:00:00	2	1	2	180	Иван Сидорович Петров	Балембин Кирилл Саныч

Рисунок 13 – Результат неявного соединения таблиц

6 Inner Join

Та же выборка, полученная с помощью оператора JOIN, представлена на рисунке 14.

```
4 • SELECT M.movie_name, S.session_date_time, S.Hall_hall_num,
5     T.ticket_row, T.ticket_seat, T.ticket_cost, CL.client_full_name, CA.cashier_full_name
6 FROM Tickets as T
7 JOIN Sessions as S ON T.Session_session_num = S.session_num
8 JOIN Movies as M ON S.Movie_movie_num = M.movie_num
9 JOIN Clients as CL ON T.Client_client_num = CL.client_num
10 JOIN Cashiers as CA ON T.Cashier_cashier_num = CA.cashier_num;
11
```

	movie_name	session_date_time	Hall_hall_num	ticket_row	ticket_seat	ticket_cost	client_full_name	cashier_full_name
▶	Майор	2024-05-06 09:00:00	3	4	5	300	Петр Иванович Сидоров	Балембин Кирилл Саныч
	Майор	2024-05-06 09:00:00	3	2	3	300	Сидор Петрович Иванов	Балембин Кирилл Саныч
	Heat	2024-05-06 13:00:00	2	1	2	180	Иван Сидорович Петров	Балембин Кирилл Саныч

Рисунок 14 – Использование оператора INNER JOIN

7 Outer Join

Та же выборка, полученная с помощью оператора LEFT JOIN (к таблице Tickets добавляются данные из других таблиц), представлена на рисунке 15.

```
4 • SELECT M.movie_name, S.session_date_time, S.Hall_hall_num,
5     T.ticket_row, T.ticket_seat, T.ticket_cost, CL.client_full_name, CA.cashier_full_name
6 FROM Tickets as T
7 LEFT JOIN Sessions as S ON T.Session_session_num = S.session_num
8 LEFT JOIN Movies as M ON S.Movie_movie_num = M.movie_num
9 LEFT JOIN Clients as CL ON T.Client_client_num = CL.client_num
10 LEFT JOIN Cashiers as CA ON T.Cashier_cashier_num = CA.cashier_num;
11
```

	movie_name	session_date_time	Hall_hall_num	ticket_row	ticket_seat	ticket_cost	client_full_name	cashier_full_name
▶	Heat	2024-05-06 13:00:00	2	1	2	180	Иван Сидорович Петров	Балембин Кирилл Саныч
	Майор	2024-05-06 09:00:00	3	2	3	300	Сидор Петрович Иванов	Балембин Кирилл Саныч
	Майор	2024-05-06 09:00:00	3	4	5	300	Петр Иванович Сидоров	Балембин Кирилл Саныч

Рисунок 15 – Использование оператора LEFT JOIN

Пример комбинированного запроса с использованием и LEFT, и RIGHT JOIN представлен на рисунке 16.

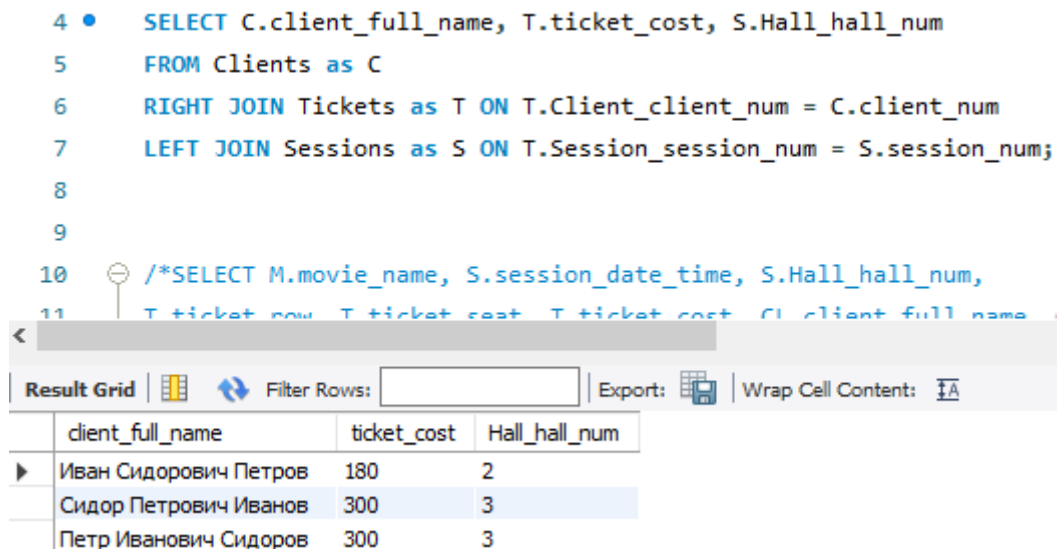


Рисунок 16 – Комбинированное использование операторов LEFT JOIN и RIGHT JOIN

8 UNION – соединение однотипных выборок

Объединение однотипных данных представлено на примере имени клиента из таблицы «Clients» и имени кассира из таблицы «Cashiers» с сортировкой по возрастанию на рисунке 17.

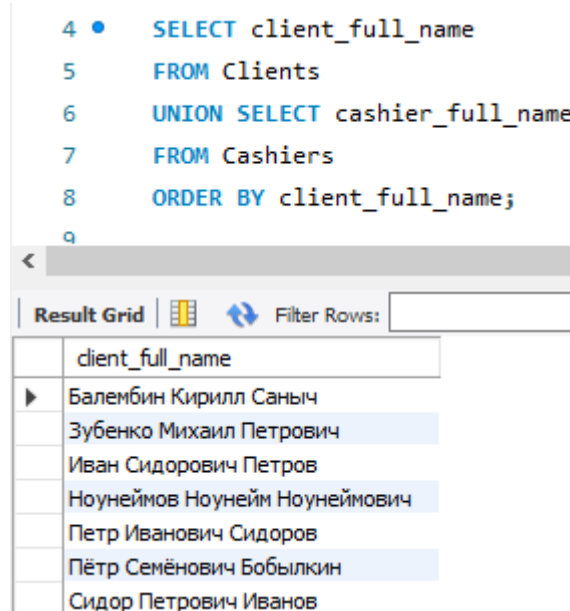


Рисунок 17 – Объединение однотипных данных разных таблиц

ВЫВОДЫ

В ходе выполнения данной работы были произведено физическое проектирование базы данных.

В ходе работы были получены полезные навыки и знания по составлению различных видов запросов к БД.

В первую очередь была рассмотрена базовая команда SELECT, позволяющая получить информацию из таблиц в заданном виде, а также вместе с ней были рассмотрены команда WHERE и логические операции, которые позволяют фильтровать данные в соответствии с нужными условиями.

После этого были изучены команды UPDATE и DELETE, которые служат соответственно для изменения и удаления данных.

Затем было рассмотрено неявное соединение таблиц (с помощью команды SELECT и логических условий) и соединение таблиц с помощью команды JOIN: внутреннее соединение (INNER), при котором выбираются указанные данные из таблиц, если заданное условие выполнено, и внешнее соединение (OUTER LEFT/RIGHT), при котором берутся все данные из одной таблицы и к ним после этого добавляются данные из другой.

Напоследок было изучено объединение однотипных данных из разных таблиц при помощи команды UNION.

Рассмотренные команды являются базовыми командами языка SQL, с помощью которых можно построить самые различные запросы к БД.